



bioinformatics.ca
bioinformaticsdotca.github.io



CC BY-SA 4.0 DEED

Attribution-ShareAlike 4.0 International

Canonical URL : <https://creativecommons.org/licenses/by-sa/4.0/>

[See the legal code](#)

You are free to:

Share — copy and redistribute the material in any medium or format for any purpose, even commercially.

Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

 **Attribution** — You must give [appropriate credit](#), provide a link to the license, and [indicate if changes were made](#). You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

 **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the [same license](#) as the original.

No additional restrictions — You may not apply legal terms or [technological measures](#) that legally restrict others from doing anything the license permits.

Notices:

You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable [exception or limitation](#).

No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as [publicity, privacy, or moral rights](#) may limit how you use the material.

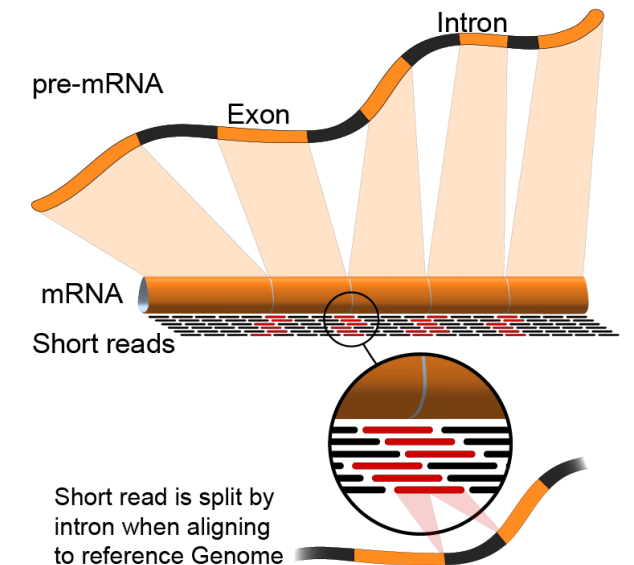
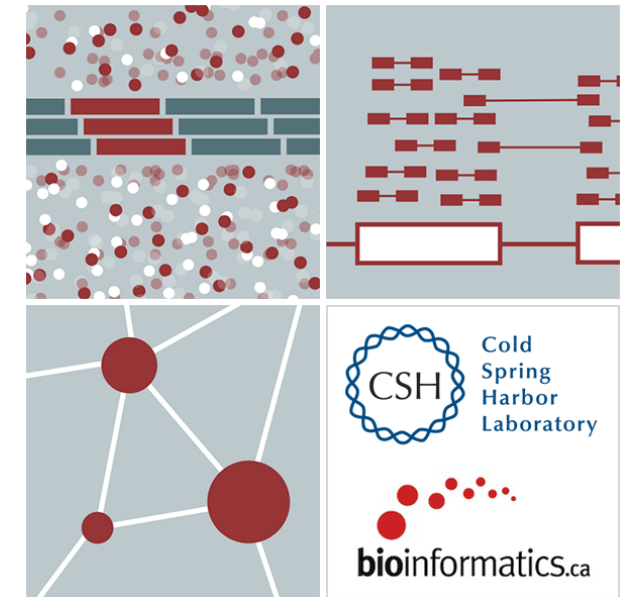


Module 2: RNA alignment and file formats

Malachi Griffith & Obi Griffith

RNA-seq Analysis

June 22-24, 2026

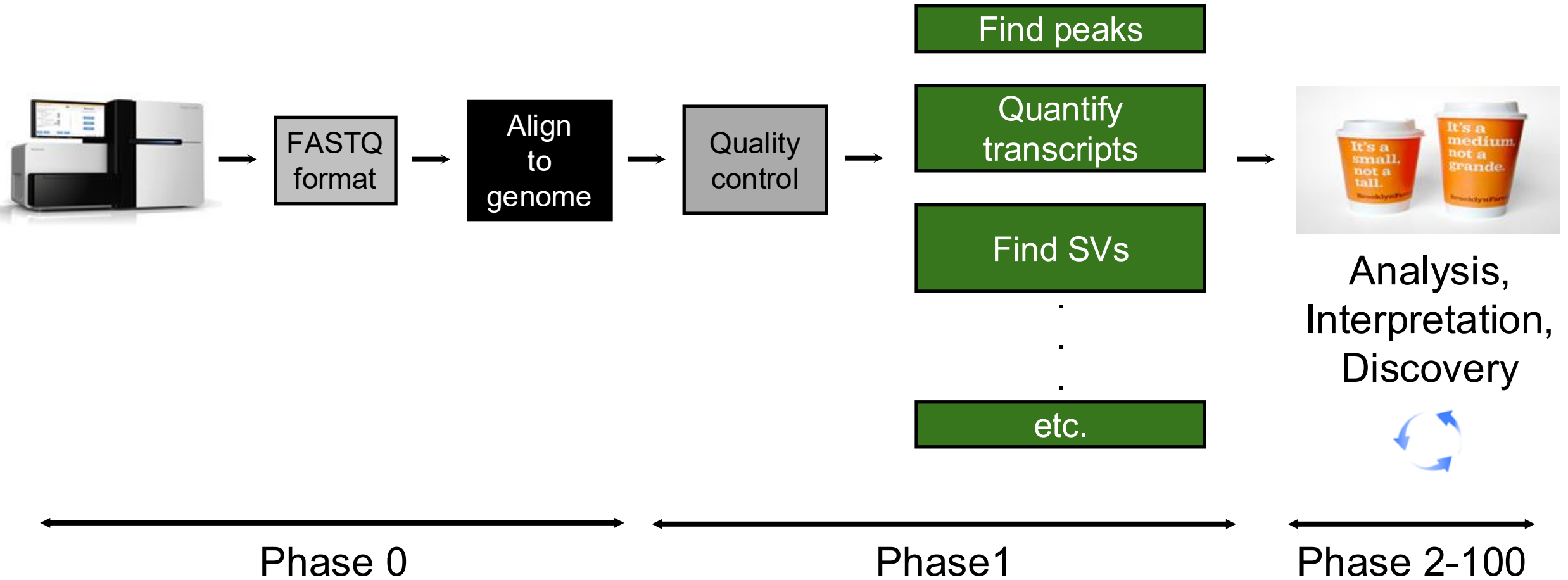




Learning objectives of module 2

- RNA-seq alignment challenges and common questions
- Alignment strategies
- HISAT2
- Introduction to the BAM and BED formats
- Basic manipulation of BAMs
- Visualization of RNA-seq alignments in IGV
- Alignment QC Assessment
- BAM read counting and determination of variant allele expression status

Alignment is central to most genomic research





Alignment - How does it work?



- Alignment is about fitting individual pieces (reads) into the correct part of the puzzle
- The human genome project gave us the picture on the box cover (the reference genome)
- Imperfections in how the pieces fit can indicate changes to a copy of the picture

Reference:

AGCCTGAGACCGTAAAAAAGTCAAG

A read sequence:

|||||||
GAGACCGTAAAAACGTC



A variant!

RNA-seq alignment challenges



- Computational cost
 - 100's of millions of reads
- Introns!
 - Align to a transcriptome or align to a genome?
 - Spliced vs. unspliced alignments
- Can I just align my data once using one approach and be done with it?
 - Unfortunately, probably not

Three RNA-seq mapping strategies



De novo assembly

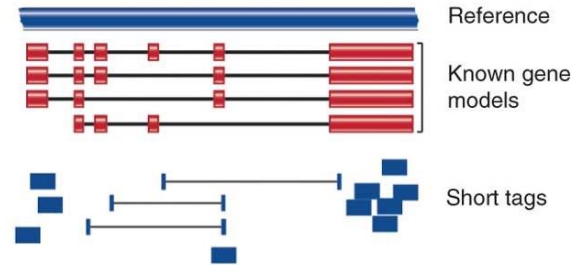


Assemble transcripts from overlapping tags



Optional: align to genome to get exon structure

Align to transcriptome



Use known and/or predicted gene models to examine individual features

Align to reference genome



Infer possible transcripts and abundance

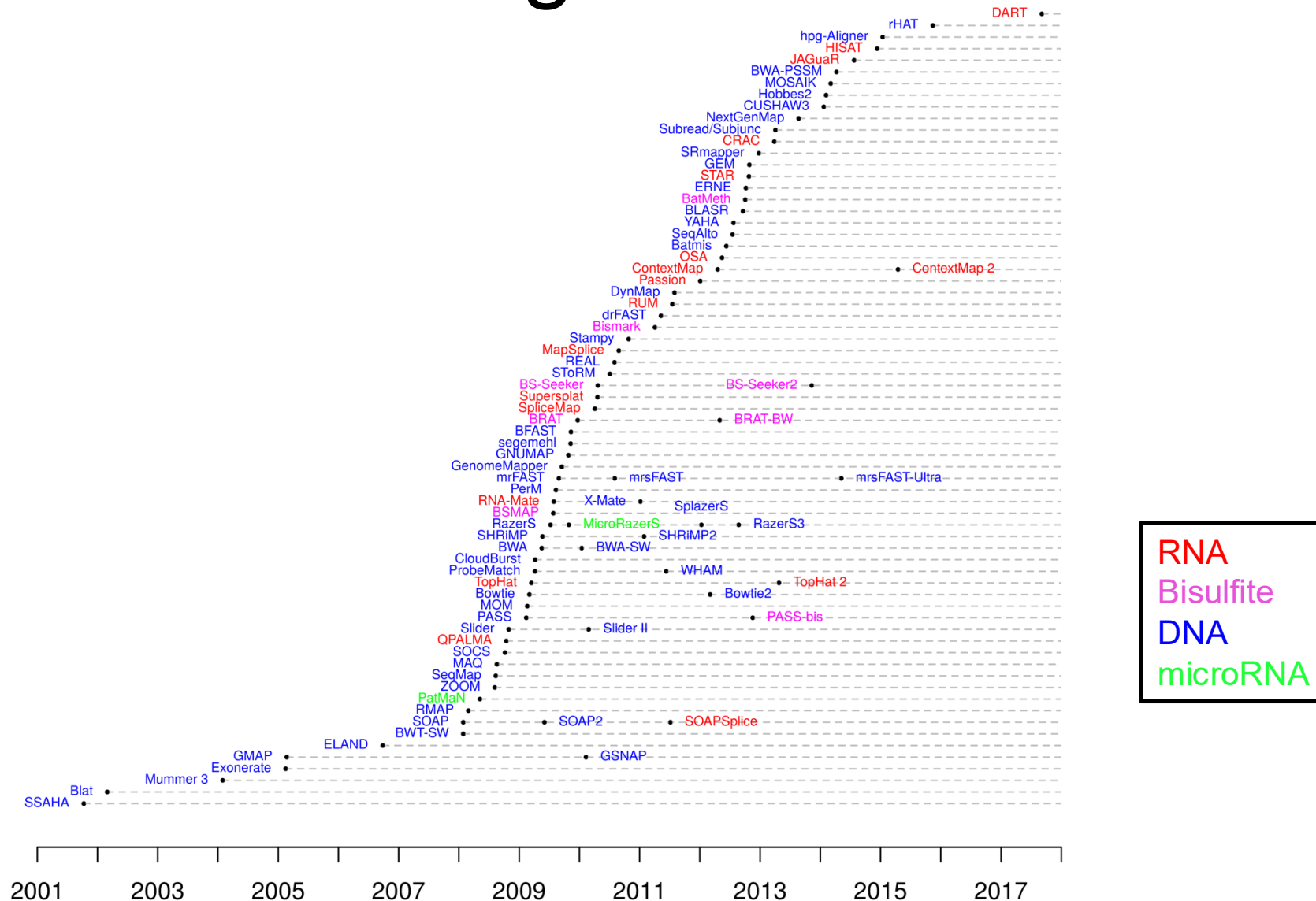
Diagrams from Cloonan & Grimmond, Nature Methods 2010

Which alignment strategy is best?



- De novo assembly
 - If a reference genome does not exist for the species being studied
 - If complex polymorphisms/mutations/haplotypes might be missed by comparing to the reference genome
- Align to transcriptome
 - If you have short reads (< 50bp)
 - Relies on known transcripts
- Align to reference genome
 - All other cases
 - Does not rely on known transcripts – allows for discovery
- Each strategy involves different alignment/assembly tools

Which read aligner should I use?



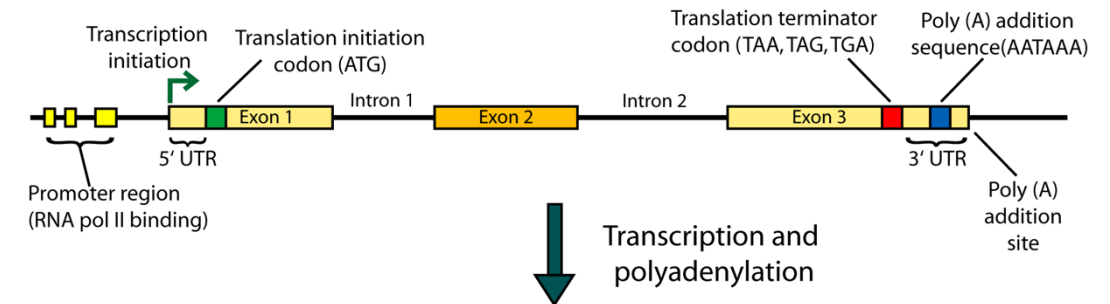
RNA
Bisulfite
DNA
microRNA

Should I use a splice-aware or unspliced mapper?

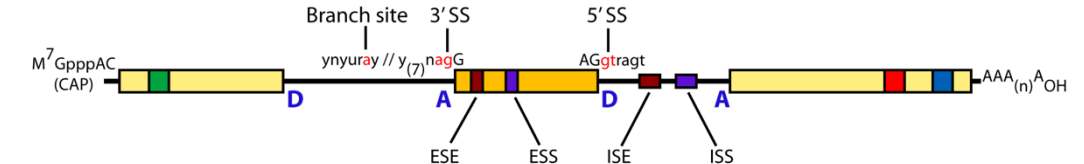


- The fragments being sequenced in RNA-seq represent mRNA - introns are removed
- But we are usually aligning these reads back to the reference genome
- Unless your reads are short (<50bp) you should use a splice-aware aligner
 - HISAT2, STAR, MapSplice, etc.

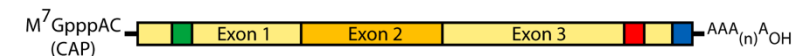
Double-stranded genomic DNA template



Single-stranded pre-mRNA (nuclear RNA)



Mature mRNA



HISAT/HISAT2



- HISAT is a 'splice-aware' RNA-seq read aligner
 - HISAT = **H**ierarchical **I**ndexing for **S**pliced **A**lignments of **T**ranscripts
- Requires a reference genome
- Very fast
- Uses an indexing scheme based on the Burrows-Wheeler transform and the Ferragina-Manzini (FM) index
- Multiple types of indexes for alignment
 - a whole-genome FM index to anchor each alignment
 - numerous local FM indexes for very rapid extensions of these alignments.
 - Whole-genome indices with SNPs and known transcript structures accounted for

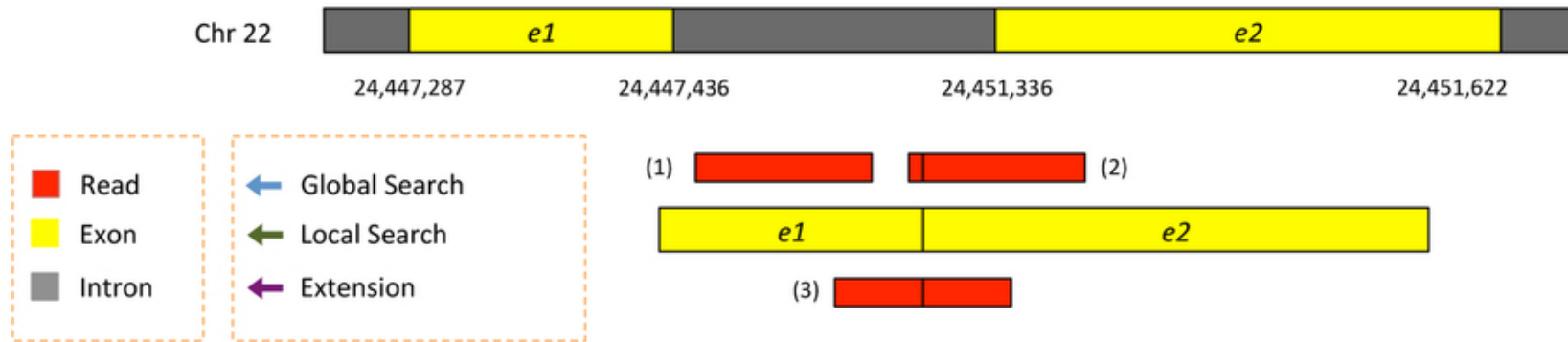
Kim et al. 2015. Nat Methods 12:357–360

HISAT/HISAT2 algorithm



- Uses a hierarchical indexing algorithm + several adaptive strategies
 - based on the position of a read with respect to splice sites
- 1) Find candidate locations across the whole genome first
 - mapping part of each read using the global FM index
 - Generally identifies one or a small number of candidates.
- 2) Do local alignment
 - selects one of ~48,000 local indexes for each candidate
 - uses it to align the remainder of the read.
- For paired reads, each mate is separately aligned
 - If a read fails to align, then the alignments of its mate are used as anchors to map the unaligned mate

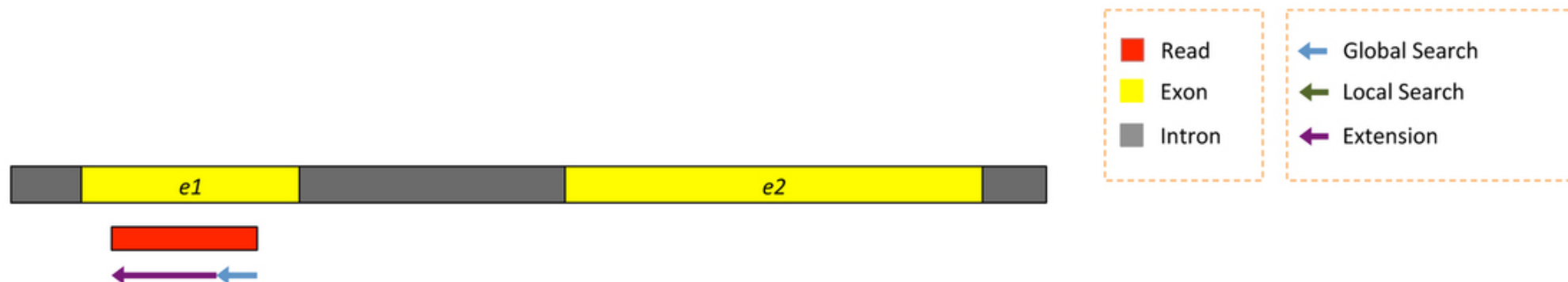
HISAT2 Alignment



- Two exons from chr22
- Three reads

Kim et al. 2015. Nat Methods 12:357–360

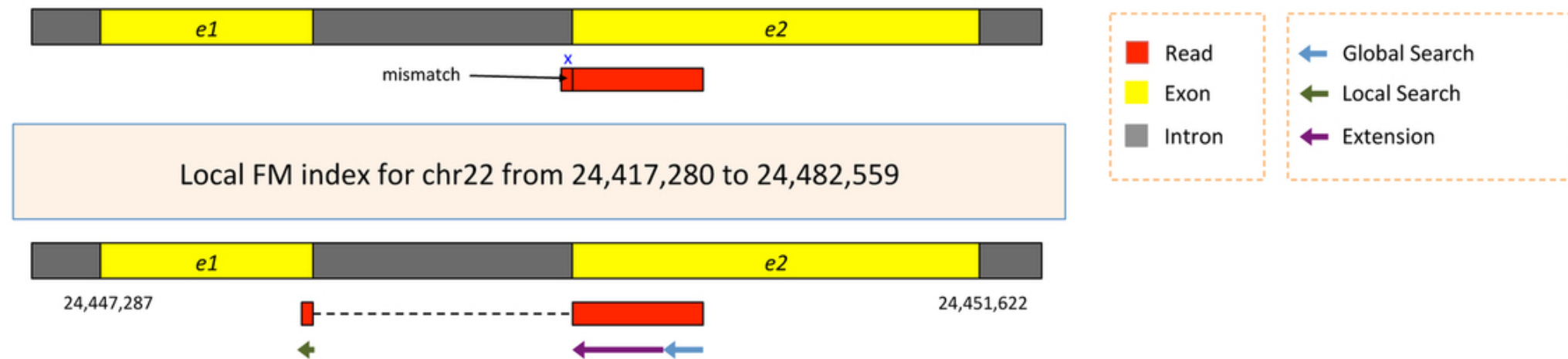
HISAT2 Alignment



- 1) Search for read position with global FM index (slower)
- 2) Once at least 28bp and exactly one location switch to extension mode against reference genome (faster)

Kim et al. 2015. Nat Methods 12:357–360

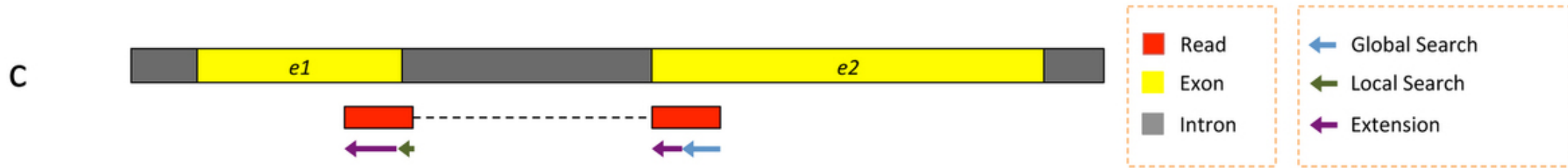
HISAT2 Alignment



- 1) Search for read position with global FM index (slower)
- 2) Extend until mismatch at 93bp (faster)
- 3) Switch to local FM index to align remaining 8bp
 - index covers only a small region, so we find just one match
- 4) Check for compatibility and combine into single spliced alignment

Kim et al. 2015. Nat Methods 12:357–360

HISAT2 Alignment



- 1) global search until exactly one match of at least 28bp (slower)
- 2) Extend until mismatch at 51bp (faster)
- 3) switch to local FM index to align first 8bp of remaining read
 - If too many matches increase prefix size
- 4) Extend again
- 5) Check for compatibility and combine into single spliced alignment

Kim et al. 2015. Nat Methods 12:357–360

What is the output of HISAT2?



- A SAM/BAM file
 - SAM stands for Sequence Alignment/Map format
 - BAM is the binary version of a SAM file
- Remember, compressed files require special handling compared to plain text files
- How can I convert BAM to SAM?
 - <http://www.biostars.org/p/1701/>
- Is HISAT2 the only mapper to consider for RNA-seq data?
 - <http://www.biostars.org/p/60478/>

Introduction to the SAM/BAM format



- The specification
 - <http://samtools.sourceforge.net/SAM1.pdf>
- SAM is uncompressed text data
- BAM is a compressed version of SAM
 - lossless BGZF format
- BAM files are usually 'indexed'
 - A '.bai' file will be found beside the '.bam' file
 - Indexing provides fast retrieval of alignments overlapping a specified region without going through all alignments.
 - BAM must be sorted by the reference ID and then the leftmost coordinate before indexing


```
mrgiffit@linux270 ~$ samtools view -H /gscmnt/gc13001/info/model_data/2891632684/build136494552/alignments/136000019.bam | grep -P "SN:22|HD|RG|PG"
@HD      VN:1.4      SO:coordinate
@SQ      SN:22      LN:51394566      UR:ftp://ftp.ncbi.nih.gov/genbank/genomes/Eukaryotes/vertebrates_mammals/Homo_sapiens/GRCh37/special_requests/GRCh37-lite.fa.gz AS:GRCh37-lite M5:a718acaa6135fdca8357d5bfe9
4211dd  SP:Homo sapiens
@RG      ID:2888721359      PL:illumina      PU:D1BA4ACXX.3      LB:H_KA-452198-0817007-cDNA-3-lib1      PI:365      DS:paired end      DT:2012-10-03T19:00:00-0500      SM:H_KA-452198-0817007      CN:WUGSC
@PG      ID:2888721359      VN:2.0.8      CL:tophat      --library-type fr-secondstrand      --bowtie-version=2.1.0
@PG      ID:MarkDuplicates      PN:MarkDuplicates      PP:2888721359      VN:1.85(exported)      CL:net.sf.picard.sam.MarkDuplicates      INPUT=/gscmnt/gc13001/info/build_merged_alignments/merged-alignment-blade10-2-5.gsc.wustl.edu-jwalker-15434-136080019/scratch-ILg6Y/H_KA-452198-0817007-cDNA-3-lib1-2888360300.bam      OUTPUT=/gscmnt/gc13001/info/build_merged_alignments/merged-alignment-blade10-2-5.gsc.wustl.edu-jwalker-15434-136080019/scratch-ILg6Y/H_KA-452198-0817007-cDNA-3-lib1-2888360300-post_dup.bam      METRICS_FILE=/gscmnt/gc13001/info/build_merged_alignments/merged-alignment-blade10-2-5.gsc.wustl.edu-jwalker-15434-136080019/staging-1iuJ5/H_KA-452198-0817007-cDNA-3-lib1-2888360300-metrics      REMOVE_DUPLICATES=false      ASSUME_SORTED=true      MAX_FILE_HANDLES_FOR_READ_ENDS_MAP=95000      TMP_DIR=/gscmnt/gc13001/info/build_merged_alignments/merged-alignment-blade10-2-5.gsc.wustl.edu-jwalker-15434-136080019/scratch-ILg6Y      VALIDATION_STRINGENCY=SILENT      MAX_RECORDS_IN_RAM=500000      PROGRAM_RECORD_ID=MarkDuplicates      PROGRAM_GROUP_NAME=Mark Duplicates      MAX_SEQUENCES_FOR_DISK_READ_ENDS_MAP=50000      SORTING_COLLECTION_SIZE_RATIO=0.25      READ_NAME_REGEX=[a-zA-Z0-9]+;[0-9];([0-9]+);([0-9]+);([0-9]+);.*      OPTICAL_DUPLICATE_PIXEL_DISTANCE=100      VERBOSITY=INFO
QUIT=false      COMPRESSION_LEVEL=5      CREATE_INDEX=false      CREATE_MD5_FILE=false
mrgiffit@linux270 ~$
```

[illegible]

SAM/BAM header section



- Used to describe source of data, reference sequence, method of alignment, etc.
- Each section begins with character '@' followed by a two-letter record type code. These are followed by two-letter tags and values:
 - @HD The header line
 - VN: format version
 - SO: Sorting order of alignments
 - @SQ Reference sequence dictionary
 - SN: reference sequence name
 - LN: reference sequence length
 - SP: species
 - @RG Read group
 - ID: read group identifier
 - CN: name of sequencing center
 - SM: sample name
 - @PG Program
 - PN: program name
 - VN: program version

A BAM file is divided into header and alignment sections



Example SAM/BAM header section

```
mgriffit@linus270 ~$ samtools view -H /gscmnt/gc13001/info/model_data/2891632684/build136494552/alignments/136080019.bam | grep -P "SN\.:22|HD|RG|PG"
@HD      VN:1.4  SO:coordinate
@SQ      SN:22  LN:51304566  UR:ftp://ftp.ncbi.nih.gov/genbank/genomes/Eukaryotes/vertebrates_mammals/Homo_sapiens/GRCh37/special_requests/GRCh37-lite.fa.gz AS:GRCh37-lite M5:a718acaa6135fdca8357d5bfe9
4211dd  SP:Homo sapiens
@RG      ID:2888721359  PL:illumina  PU:D1BA4ACXX.3  LB:H_KA-452198-0817007-cDNA-3-lib1  PI:365  DS:paired end  DT:2012-10-03T19:00:00-0500  SM:H_KA-452198-0817007  CN:WUGSC
@PG      ID:2888721359  VN:2.0.8  CL:tophat --library-type fr-secondstrand --bowtie-version=2.1.0
@PG      ID:MarkDuplicates  PN:MarkDuplicates  PP:2888721359  VN:1.85(exported)  CL:net.sf.picard.sam.MarkDuplicates INPUT=[/gscmnt/gc13001/info/build_merged_alignments/merged-alignment-blade10-2-5.gsc.wustl.edu-jwalker-15434-136080019/scratch-ILg6Y/H_KA-452198-0817007-cDNA-3-lib1-2888360300.bam] OUTPUT=/gscmnt/gc13001/info/build_merged_alignments/merged-alignment-blade10-2-5.gsc.wustl.edu-jwalker-15434-136080019/scratch-ILg6Y/H_KA-452198-0817007-cDNA-3-lib1-2888360300-post_dup.bam METRICS_FILE=/gscmnt/gc13001/info/build_merged_alignments/merged-alignment-blade10-2-5.gsc.wustl.edu-jwalker-15434-136080019/staging-1iuJS/H_KA-452198-0817007-cDNA-3-lib1-2888360300.metrics REMOVE_DUPLICATES=false ASSUME_SORTED=true MAX_FILE_HANDLES_FOR_READ_ENDS_MAP=9500 TMP_DIR=[/gscmnt/gc13001/info/build_merged_alignments/merged-alignment-blade10-2-5.gsc.wustl.edu-jwalker-15434-136080019/scratch-ILg6Y] VALIDATION_STRINGENCY=SILENT MAX_RECORDS_IN_RAM=500000 PROGRAM_RECORD_ID=MarkDuplicates PROGRAM_GROUP_NAME=MarkDuplicates MAX_SEQUENCES_FOR_DISK_READ_ENDS_MAP=50000 SORTING_COLLECTION_SIZE_RATIO=0.25 READ_NAME_REGEX=[a-zA-Z0-9]+:[0-9]:([0-9]+):([0-9]+):([0-9]+):([0-9]+).* OPTICAL_DUPLICATE_PIXEL_DISTANCE=100 VERBOSITY=INFO QUIET=false COMPRESSION_LEVEL=5 CREATE_INDEX=false CREATE_MD5_FILE=false
mgriffit@linus270 ~$
```

Version (VN) and sort order (SO) - Important!

Reference sequence (SQ) and sequence length (LN)

```
@HD      VN:1.3  SO:coordinate
@SQ      SN:20  LN:63025520
@RG      ID:HG00096  SM:HG00096
@PG      ID:HG00096  PN:bwa  CL:/Users/AlistairNWard/Work/gkno/gkno_launcher/tools/bwa/bwa mem -t 4
```

Read group (RG) and sample (SM)

Programs (PG) that have been run on the data

[illegible]

Col	Field	Type	Regex/Range	Brief description
1	QNAME	String	[!-?A-~]{1,255}	Query template NAME
2	FLAG	Int	[0,2 ¹⁶ -1]	bitwise FLAG
3	RNAME	String	* [!-()+-<>-~] [!-~]*	Reference sequence NAME
4	POS	Int	[0,2 ²⁹ -1]	1-based leftmost mapping POSition
5	MAPQ	Int	[0,2 ⁸ -1]	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHPX=])+	CIGAR string
7	RNEXT	String	* = [!-()+-<>-~] [!-~]*	Ref. name of the mate/next segment
8	PNEXT	Int	[0,2 ²⁹ -1]	Position of the mate/next segment
9	TLEN	Int	[-2 ²⁹ +1,2 ²⁹ -1]	observed Template LENgth
10	SEQ	String	* [A-Za-z=.]+	segment SEQUENCE
11	QUAL	String	[!-~]+	ASCII of Phred-scaled base QUALity+33

Example values

1	QNAME e.g. HWI-ST495_129147882:1:2302:10269:12362
2	FLAG e.g. 99
3	RNAME e.g. 1
4	POS e.g. 11623
5	MAPQ e.g. 3
6	CIGAR e.g. 100M
7	RNEXT e.g. =
8	PNEXT e.g. 11740
9	TLEN e.g. 217
10	SEQ e.g. CCTGTTTCTCCACAAAGTGTTTACTTTTGGATTTTGGCAGTCTAACAGGTGAAGCCCTGGAGATTCTTATTAGTGATTTGGGCTGGGGCCTGGCCATGT
11	QUAL e.g. CCCFFFFFFHHHHHJJJFIJJJJJJJJJJHJJJJJJJJJJGHHJJJJJJJJJJGHGGJJJJJJJEHHHHFFFFCDCDDDDDDDB@ACDD

SAM Format – Information Fields



Col	Field	Type	Regex/Range	Brief description
1	QNAME	String	[!-?A-~]{1,255}	Query template NAME
2	FLAG	Int	[0,2 ¹⁶ -1]	bitwise FLAG
3	RNAME	String	* [!-()+-<>-~] [!-~]*	Reference sequence NAME
4	POS	Int	[0,2 ³¹ -1]	1-based leftmost mapping POSition
5	MAPQ	Int	[0,2 ⁸ -1]	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHPX=])+	CIGAR string
7	RNEXT	String	* = [!-()+-<>-~] [!-~]*	Ref. name of the mate/next read
8	PNEXT	Int	[0,2 ³¹ -1]	Position of the mate/next read
9	TLEN	Int	[-2 ³¹ +1,2 ³¹ -1]	observed Template LENgth
10	SEQ	String	* [A-Za-z=.]+	segment SEQuence
11	QUAL	String	[!-~]+	ASCII of Phred-scaled base QUALity+33

1 2 3 4 5 6 7 8 9 10

SRR062634.14576120 163 20 899919 60 100M = 900037 218 TTCCCCAGTAGCTGGGATTACAGGCATACGCCACCATC

?

?

SAM/BAM flags explained



- 12 bitwise flags describing the alignment
- Stored as a binary string of length 12 instead of 12 columns of data
- Value of '1' indicates the flag is set. e.g. 001000000000
- All combinations can be represented as a number from 0 to 4095 (i.e. 2^{12}). This number is used in the BAM/SAM file.
- You can specify 'required' or 'filter' flags in samtools view using the '-f' and '-F' options respectively

Bit		Description
1	0x1	template having multiple segments in sequencing
2	0x2	each segment properly aligned according to the aligner
4	0x4	segment unmapped
8	0x8	next segment in the template unmapped
16	0x10	SEQ being reverse complemented
32	0x20	SEQ of the next segment in the template being reverse complemented
64	0x40	the first segment in the template
128	0x80	the last segment in the template
256	0x100	secondary alignment
512	0x200	not passing filters, such as platform/vendor quality controls
1024	0x400	PCR or optical duplicate
2048	0x800	supplementary alignment

Note that to maximize confusion, each bit is described in the SAM specification using its hexadecimal representation (i.e., '0x10' = 16 and '0x40' = 64).

<http://broadinstitute.github.io/picard/explain-flags.html>

SAM Format – Information Fields



Col	Field	Type	Regex/Range	Brief description
1	QNAME	String	[!-?A-~]{1,255}	Query template NAME
2	FLAG	Int	[0,2 ¹⁶ -1]	bitwise FLAG
3	RNAME	String	* [!-()+-<>-~] [!-~]*	Reference sequence NAME
4	POS	Int	[0,2 ³¹ -1]	1-based leftmost mapping POSition
5	MAPQ	Int	[0,2 ⁸ -1]	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHPX=])+	CIGAR string
7	RNEXT	String	* = [!-()+-<>-~] [!-~]*	Ref. name of the mate/next read
8	PNEXT	Int	[0,2 ³¹ -1]	Position of the mate/next read
9	TLEN	Int	[-2 ³¹ +1,2 ³¹ -1]	observed Template LENgth
10	SEQ	String	* [A-Za-z=.]+	segment SEQuence
11	QUAL	String	[!-~]+	ASCII of Phred-scaled base QUALity+33

1	2	3	4	5	6	7	8	9	10
SRR062634.14576120	163	20	899919	60	100M	=	900037	218	TTCCCCAGTAGCTGGGATTACAGGCATACGCCACCATC
	?				?				

CIGAR strings explained



- The 'CIGAR' (Compact Idiosyncratic Gapped Alignment Report)
- The CIGAR string is a sequence of base lengths and associated 'operations' indicating which bases align to the reference (either a match or mismatch), are deleted, are inserted, represent introns, etc.

Op	BAM	Description
M	0	alignment match (can be a sequence match or mismatch)
I	1	insertion to the reference
D	2	deletion from the reference
N	3	skipped region from the reference
S	4	soft clipping (clipped sequences present in SEQ)
H	5	hard clipping (clipped sequences NOT present in SEQ)
P	6	padding (silent deletion from padded reference)
=	7	sequence match
X	8	sequence mismatch

•e.g. 81M859N19M

•A 100 bp read consists of: 81 bases of alignment to reference, 859 bases skipped (an intron), 19 bases of alignment

CRAM files



- CRAM is an ultra-compressed version of a BAM file
 - Usually between 30-60% smaller than the corresponding BAM
- Stores “diffs” from the reference genome
 - requires the matching reference genome to restore original data!
- Base quality binning may be used as well
- Some tools still require conversion back to bam

Quality Score Bins	Example of Empirically Mapped Quality Scores*
N (no call)	N (no call)
2–9	6
10–19	15
20–24	22
25–29	27
30–34	33
35–39	37
≥ 40	40

By replacing the quality scores between 19 and 25 with a new score of 22, data storage space is conserved.

*The mapped quality score of each bin (except “N”) is subject to change depending on individual Q-tables.

How should I sort my SAM/BAM file?



- Generally BAM files are sorted by position
 - This is for performance reasons
 - When sorted and indexed, arbitrary positions in a massive BAM file can be accessed rapidly
- Certain tools require a BAM sorted by read name
 - Usually this is when we need to easily identify both reads of a pair
 - The insert size between two reads may be large
 - In fusion detection we are interested in read pairs that map to different chromosomes

Introduction to the BED format



- When working with BAM files, it is very common to want to examine a focused subset of the reference genome
 - e.g. the exons of a gene
- These subsets are commonly specified in 'BED' files
 - <https://genome.ucsc.edu/FAQ/FAQformat.html#format1>
- Many BAM manipulation tools accept regions of interest in BED format
- Basic BED format (tab separated):
 - Chromosome name, start position, end position (BED3)
 - Coordinates in BED format are 0 based

Introduction to the BED format



- There are several flavors of BED format: BED3, BED4, BED6, BED8, etc
- First 3 fields always required: chr, start, stop
- Followed by up to 9 additional optional fields: name, score, strand, thickStart, thickEnd, itemRGB, blockCount, blockSizes, blockStarts

chr7	127471196	127472363	Pos1	0	+
chr7	127472363	127473530	Pos2	0	+
chr7	127473530	127474697	Pos3	0	+
chr7	127474697	127475864	Pos4	0	+
chr7	127475864	127477031	Neg1	0	-
chr7	127477031	127478198	Neg2	0	-
chr7	127478198	127479365	Neg3	0	-
chr7	127479365	127480532	Pos5	0	+
chr7	127480532	127481699	Neg4	0	-

Manipulation of SAM/BAM and BED files



- Several tools are used ubiquitously in sequence analysis to manipulate these files
- SAM/BAM files
 - samtools
 - bamtools
 - Picard
- BED files
 - bedtools
 - bedops

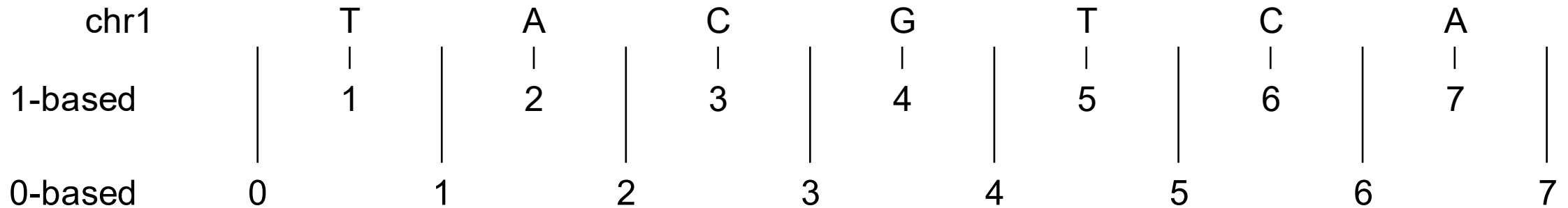


Common sources of confusion



- Genomic coordinate systems
- Genome builds
- Variant representation

Genomic coordinates – 1 vs 0 based



	1-based	0-based
Indicate a single nucleotide	chr1:4-4 G	chr1:3-4 G
Indicate a range of nucleotides	chr1:2-4 ACG	chr1:1-4 ACG
Indicate a single nucleotide variant	chr1:5-5 T/A	chr1:4-5 T/A

- 1-based : Single nucleotides, variant positions, or ranges are specified directly by their corresponding nucleotide numbers
 - GFF, SAM, VCF, Ensembl browser, ...
- 0-based: Single nucleotides, variant positions, or ranges are specified by the coordinates that flank them
 - BED, BAM, UCSC browser, ...

Genome builds



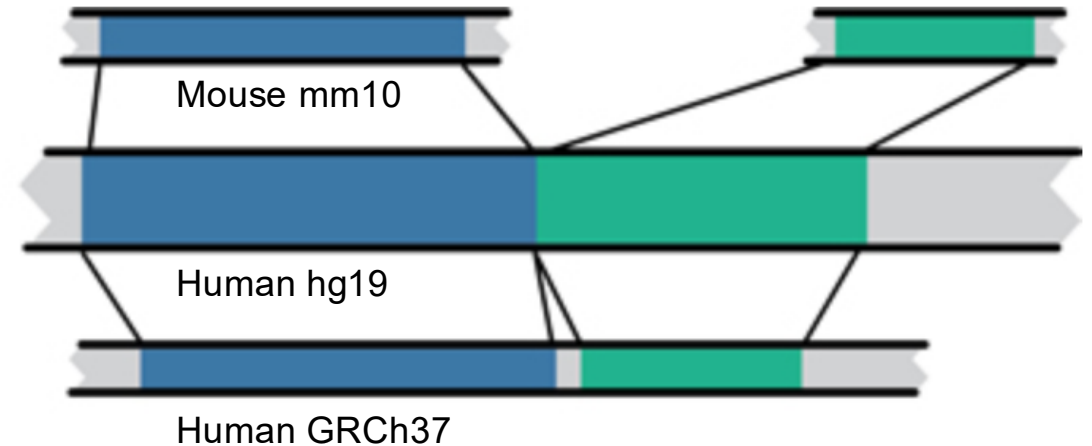
Reference Genome builds

Current human: GRCh38, hg38, b38
alternates: GRCh38v2_ccdg,
GRCh38_full_analysis_set_plus_decoy_hla

Previous human: GRCh37, hg19, b37

Current mouse: GRCm38, mm10

Lift-over



For a detailed discussion of various human reference genome flavors refer here:
https://pmbio.org/module-02-inputs/0002/02/01/Reference_Genome/

Variant shifting (alignment) and parsimony/trimming



Reference and alternative alleles of a CA short tandem repeat (STR)

REF
ALT

GGGCACACACAGGG
GGGCACACAGGG

← CA deletion from the reference

Genome Reference		Variant Call Format			
GGGCACACACAGGG		POS	REF	ALT	
REF	CA	8	CA	.	Not left aligned and alternate allele is empty
ALT	.				
REF	CAC	6	CAC	C	Not left aligned but parsimonious
ALT	C				
REF	GCACA	3	GCACA	GCA	Not right trimmed
ALT	GCA				
REF	GGCA	2	GGCA	GG	Not left trimmed
ALT	GG				
REF	GCA	3	GCA	G	Normalized (left aligned & parsimonious)
ALT	G				

Alleles represented against the human genome reference. Allele pairs are colored the same, all are representations of the same variant.

Alleles represented in Variant Call Format, all are representations of the same variant.

Parsimony: representing variant in as few nucleotides as possible without reducing the length of any allele to 0

Left (right) aligning = shifting the start position of a variant as far to the left (right) as possible



Coffee Break & Networking Session

Workshop Sponsors

